

Memory Hierarchy

Adiesha Liyana Ralalage

Memory Hierarchy

- What are the design constraints on a computer's memory
 - How much?
 - If the capacity is there, applications will likely be developed to use it
 - How fast?
 - Memory must be able to keep up with the processor
 - How expensive?
 - Cost of memory must be reasonable in relationship to the other components
- There is a trade-off between these things.

```
graph LR; A[Faster access time = greater cost per bit] --> B[Greater capacity = smaller cost per bit]; B --> C[Greater capacity = slower access speed];
```

Faster
access time
= greater
cost per bit

Greater capacity =
smaller cost per bit

Greater capacity
= slower access
speed

Memory Hierarchy

- What is the solution?
 - A memory hierarchy

Memory Hierarchy

- Going down the hierarchy
 - A. Decreasing cost per bit
 - B. Increasing capacity
 - C. Increasing access time
 - D. Decreasing frequency of access to the memory by the processor
- You might think having this kind of hierarchy may not improve the access time.
- Let's look at a calculation.

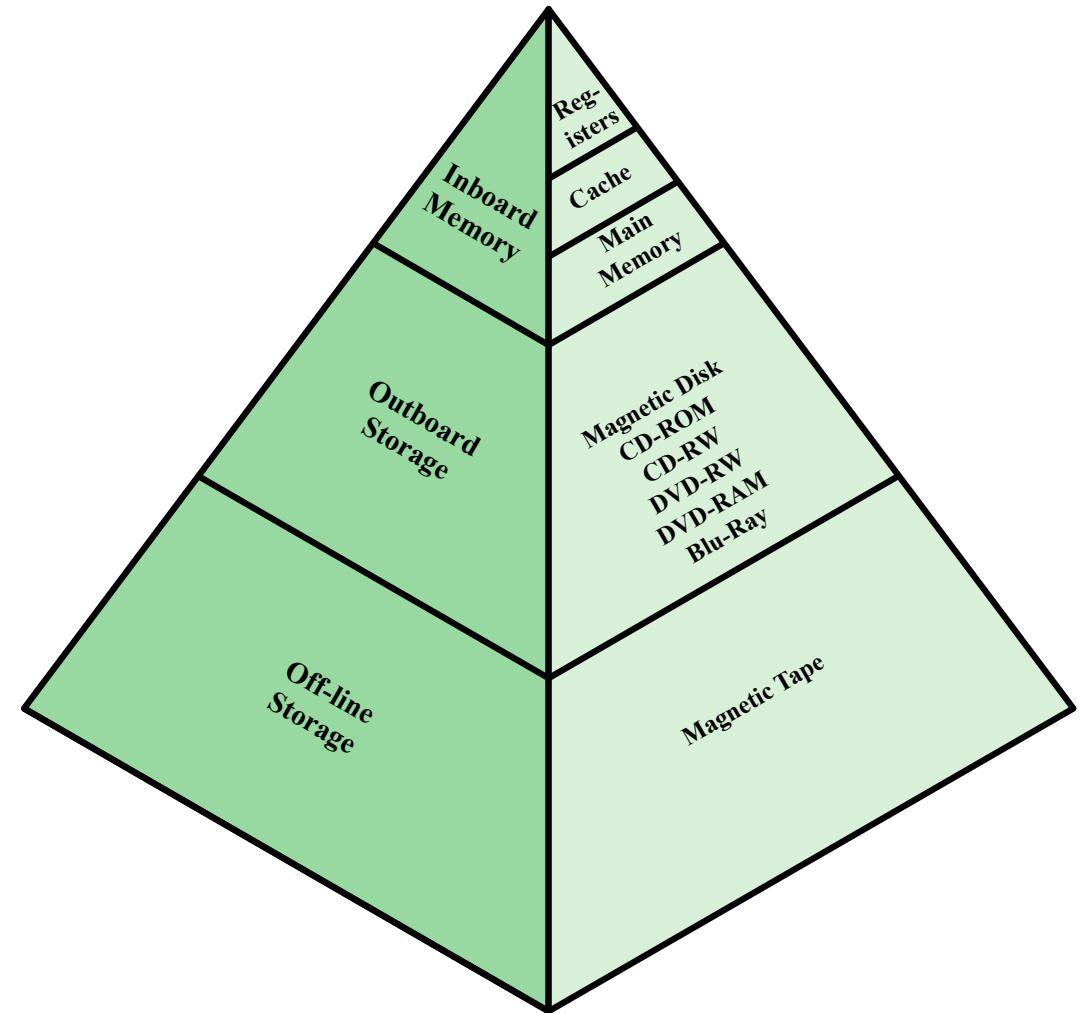


Figure 1.14 The Memory Hierarchy

Hit Ratio (H) and Average Access Time (A)

- Hypothetical memory system
 - Two levels
 - Level 01: 1000 Bytes: $0.1 \mu s$ access time.
 - Level 02: 100000 Bytes: $1 \mu s$ access time.
 - $H = \text{fraction of memory accesses that are found in fastest memory}$
- What is average access time as a function of the above parameters?
 - $A = H \cdot (0.1 \mu s) + (1 - H) \cdot (1 \mu s + 0.1 \mu s)$
- If $H = 0.95$, then
 - $A = 0.15 \mu s = 0.95 \cdot (0.1 \mu s) + (0.05) \cdot (1 \mu s + 0.1 \mu s)$

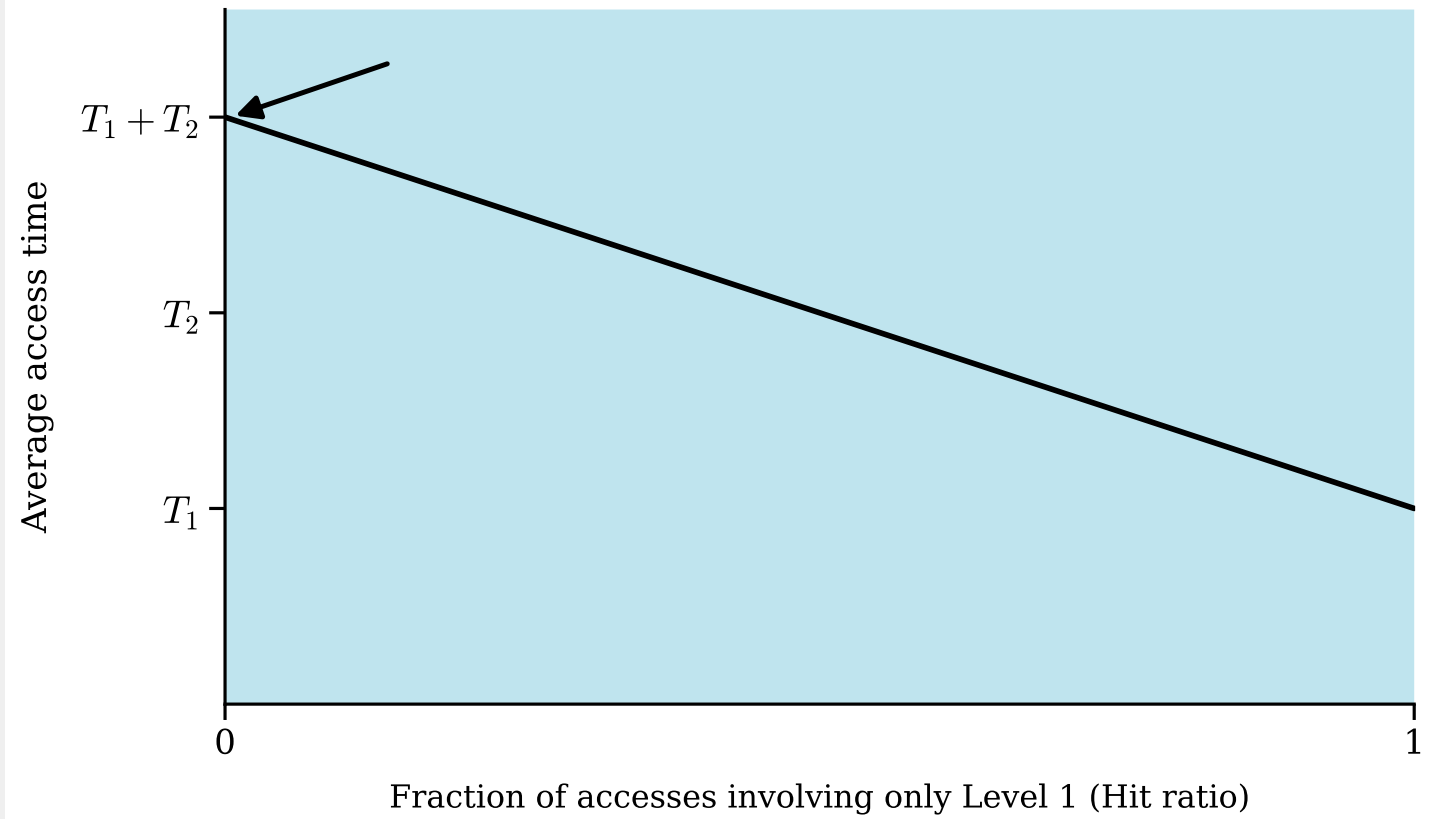


Figure 1.15 Performance of a Simple Two-Level Memory

- As long as the conditions A-D that we looked earlier holds, the average access time is closer to the fastest memory access time.
- Condition D is generally valid.

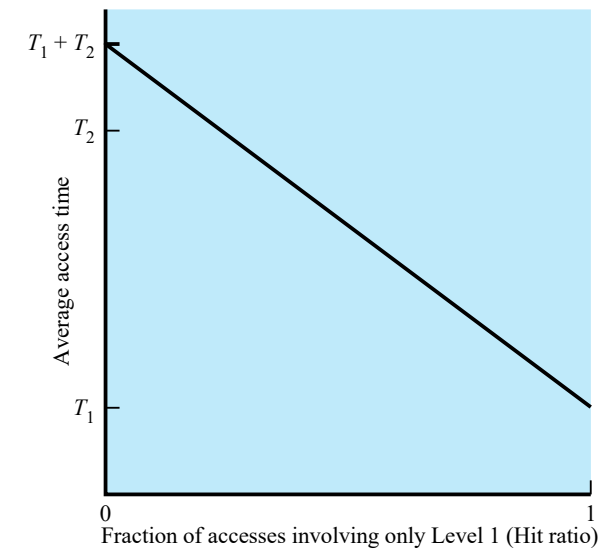


Figure 1.15 Performance of a Simple Two-Level Memory

Principle of Locality

- Memory reference by the processor tend to cluster.
- Data is organized so that the percentage of accesses to each successively lower level is substantially less than that of the levels above.
- Can be applied across more than two levels of memory

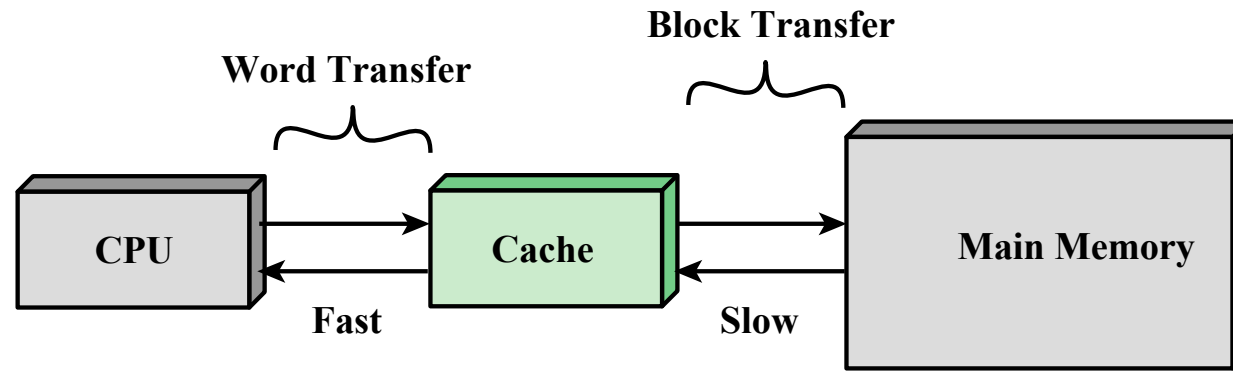
- Program execution is mostly sequential.
- For short periods of time, references to instructions tends to be localized.
- Many programs involves processing data structures such as arrays or sequential records which tends to cluster together.

Secondary memory/Auxiliary memory

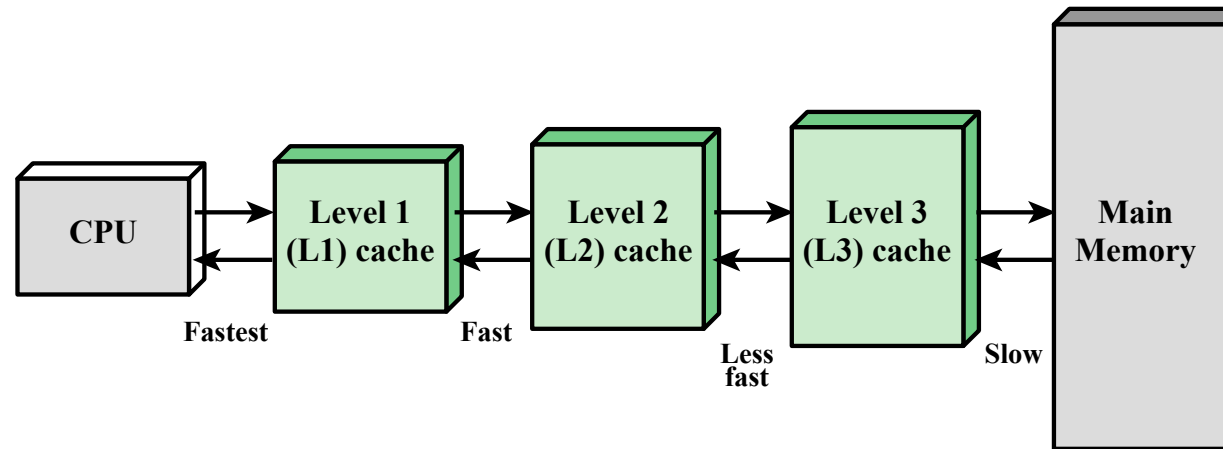
- Nonvolatile.
- External.
- Used to store program and data.
- Less expensive.

Cache Memory

- Largely invisible to the OS
- Interacts with other memory management hardware
- Processor must access memory at least once per instruction cycle
- Processor execution is limited by memory cycle time
- Exploit the principle of locality with a small, fast memory



(a) Single cache



(b) Three-level cache organization

Figure 1.16 Cache and Main Memory

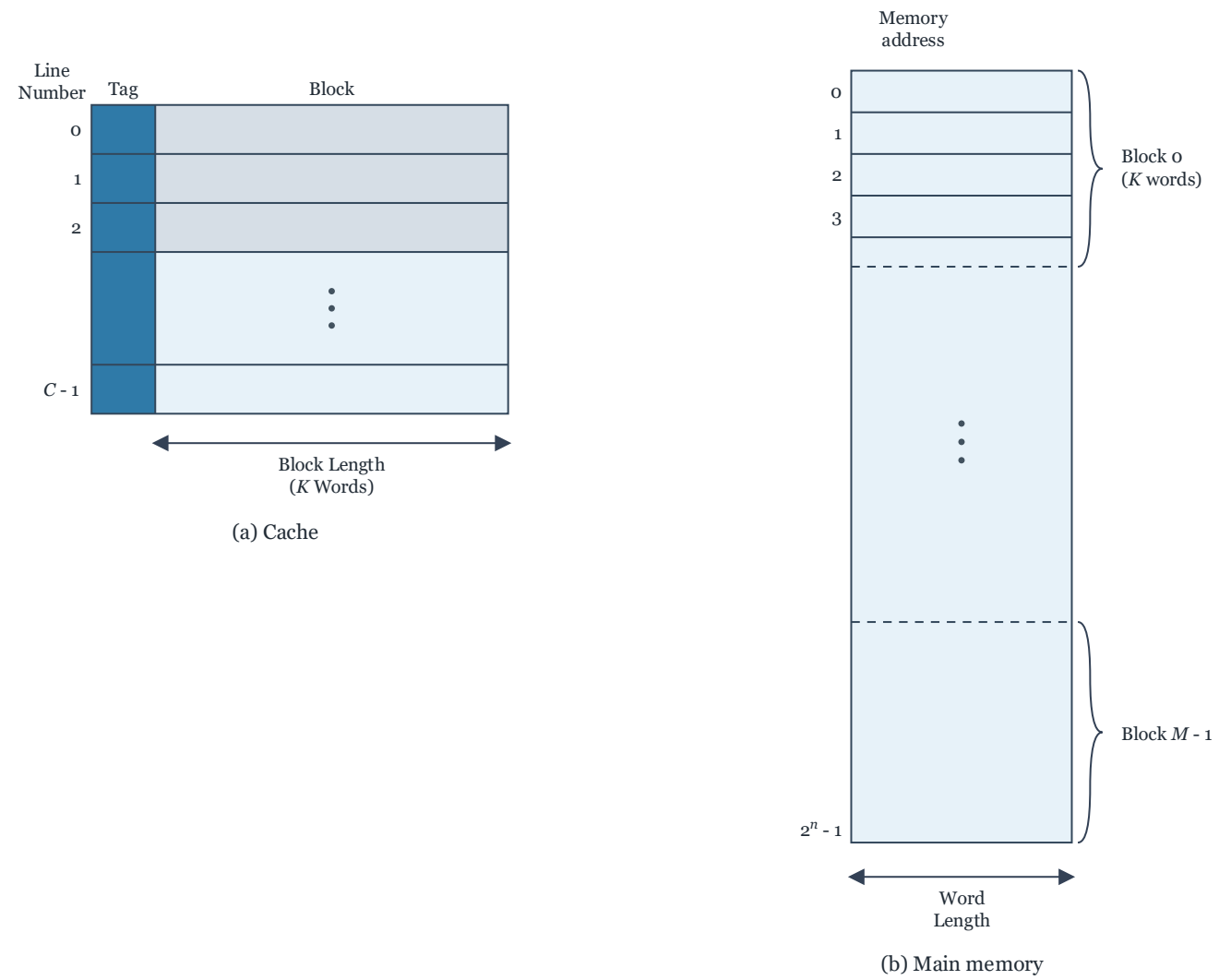


Figure 1.17 Cache/Main-Memory Structure

Let's checkout an animation



cache-memory-structure.html

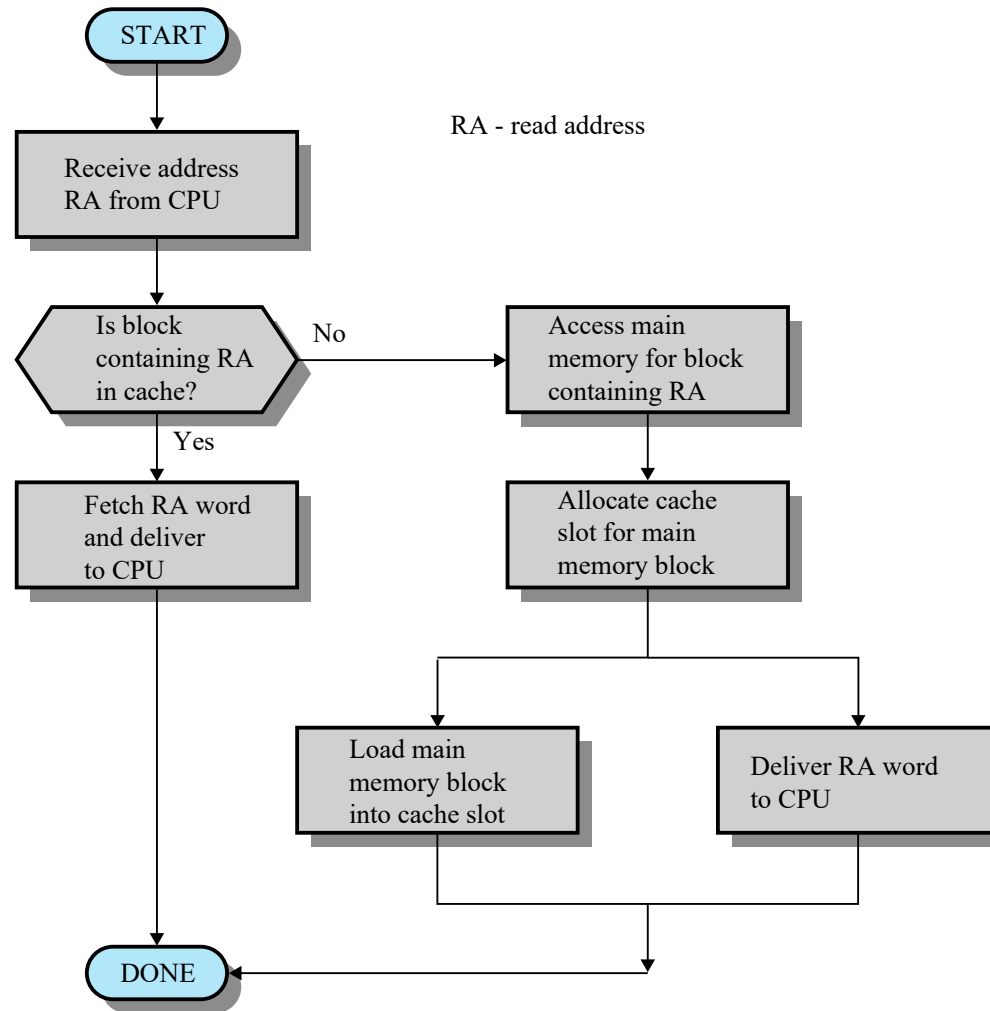


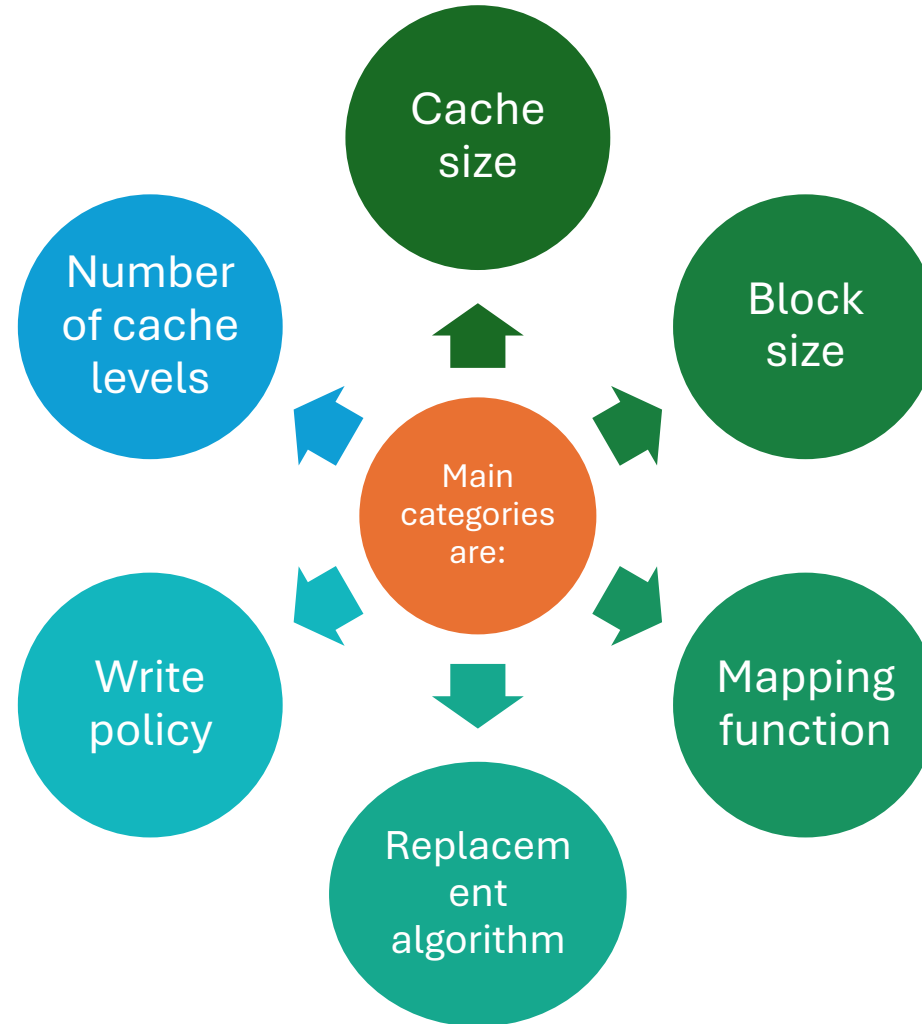
Figure 1.18 Cache Read Operation

Animation



cache-read-operation.html

Cache Design



Cache size and Block size

Cache Size

Small caches
have significant
impact on
performance

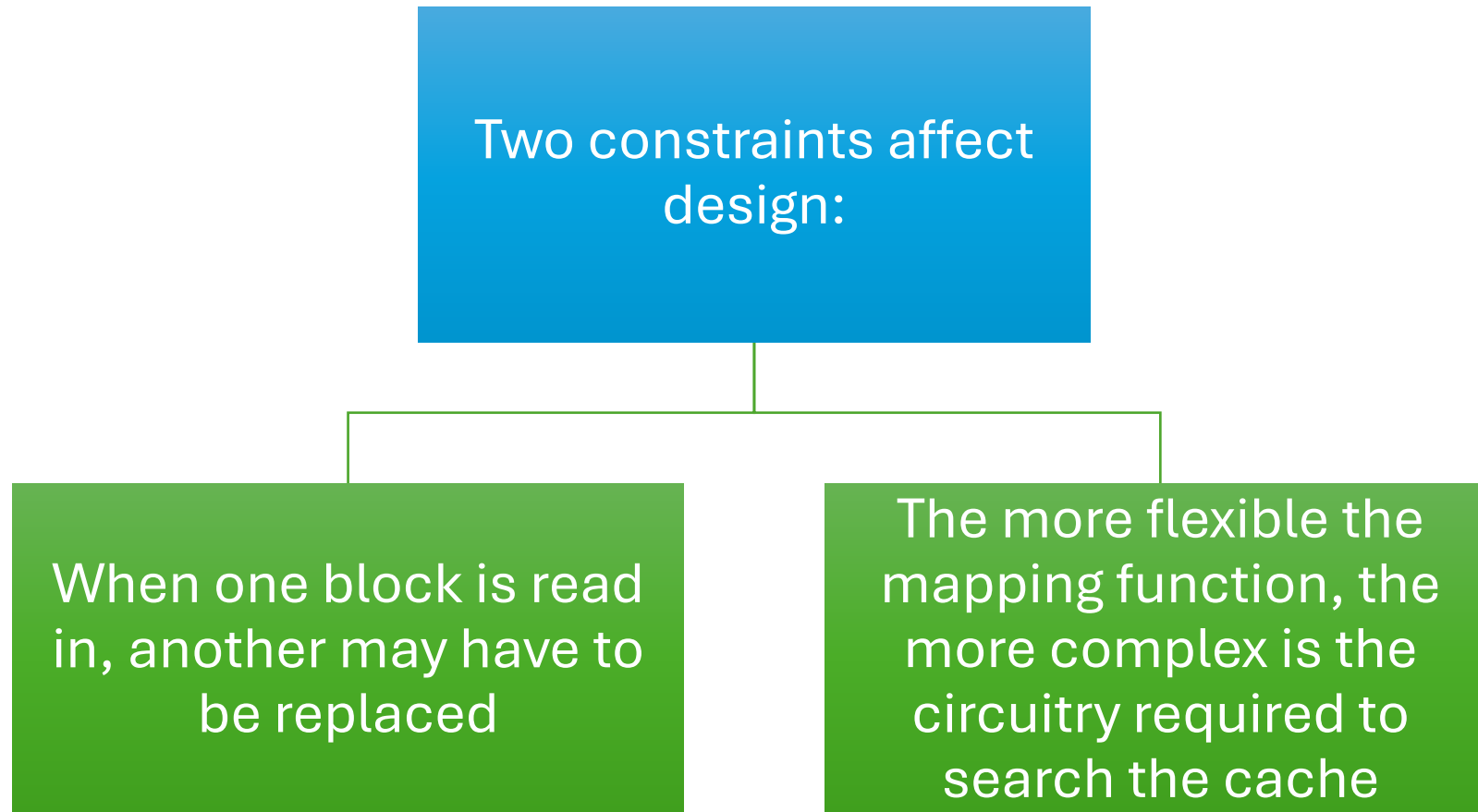
Block
Size

The unit of data
exchanged
between cache
and main memory

- **Increasing block size initially improves the hit ratio** because each transfer brings in more nearby data.
- **This exploits spatial locality:** nearby data are likely to be accessed soon.
- **If the block becomes too large, the hit ratio decreases** because much of the newly fetched data may not be used.
- **Large blocks also evict useful data** from the limited cache, reducing the chance of future cache hits.
- **Therefore, there is a “sweet spot” for block size** that provides a good balance between exploiting locality and avoiding unnecessary data transfers and evictions.

Mapping function

- Determines which cache location the block will occupy.



1. Direct Mapping

- **Each memory block has exactly one possible cache location.**
- Ex: *Cache location = Memory block number mod 10*
- $37 \bmod 10 = 7$

2. Fully Associative Mapping

- **A memory block can go anywhere in the cache.**
- Ex: Block 37 can fit into any slot
- **Advantage:** Very flexible; fewer conflicts.
- **Disadvantage:** The cache has to search/compare against many entries to determine whether the requested block is present, making the hardware more complex.

3. Set Associative Mapping

- This is a **compromise between the two**.
- Divide the cache into **sets**. A memory block is assigned to **one particular set**, but it can go into **any slot within that set**.
- For example, suppose we have 10 cache slots organized into 5 sets, with 2 slots per set:
 - Block #37 might be assigned to Set 2.

Replacement algorithm

- Least Recently Used (LRU) Algorithm
 - Effective strategy is to replace a block that has been in the cache the longest with no references to it
 - Hardware mechanisms are needed to identify the least recently used block
 - Chooses which block to replace when a new block is to be loaded into the cache

Write Policy

Dictates when the memory write operation takes place

- Can occur every time the block is updated
- Can occur when the block is replaced
 - Minimizes write operations
 - Leaves main memory in an obsolete state

Write Policy

- Could there a scenario where updating main memory after each update in the cache is beneficial?
 - Multiple processors may share main memory and write-through ensures all processors see consistent data without needing complex coherence protocols.

Analysis of Two-level memory

- Level 1 (M_1): faster, expensive, smaller
 - T_1 : access time for M_1
 - C_1 : average cost per bit for M_1
 - S_1 : size of M_1
- Level 2 (M_2): slower, cheaper, larger
 - T_2 : access time for M_2
 - C_2 : average cost per bit for M_2
 - S_2 : size of M_2

- $T_s = H \times T_1 + (1 - H) \times (T_1 + T_2) = T_1 + (1 - H) \times T_2$

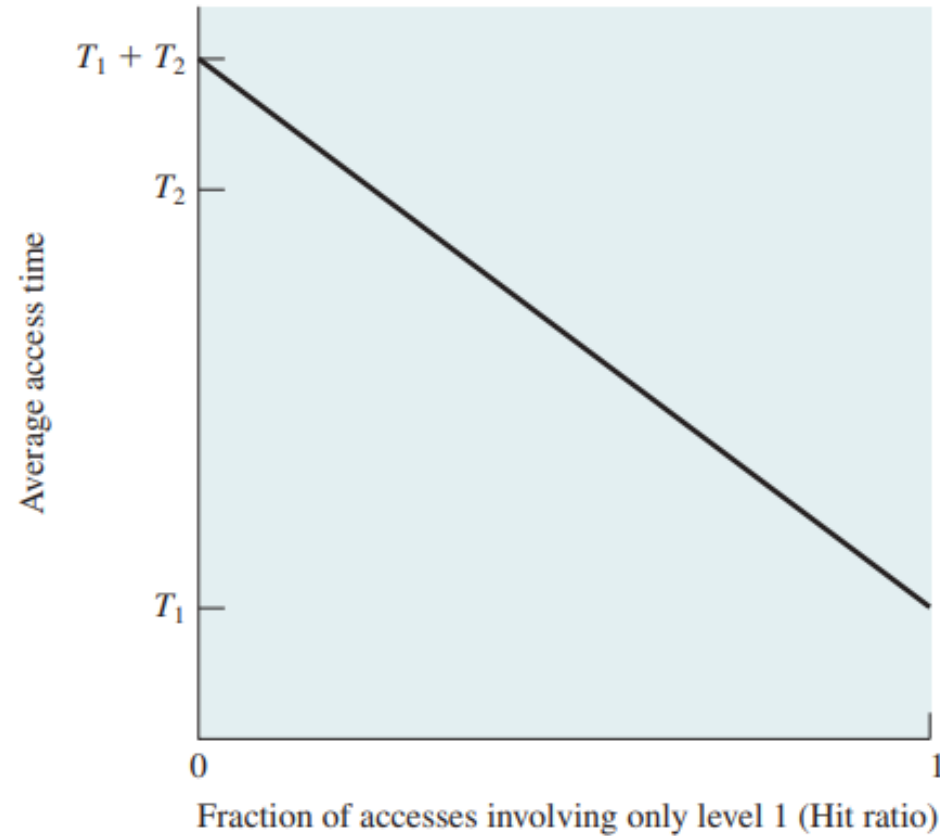


Figure 1.15 Performance of a Simple Two-Level Memory

Average cost per bit

- $C_s = \text{Average cost per bit for the combined two level memory.}$
- $C_s = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2},$
- We would like $C_s \approx C_2$
 - Given that $C_1 \gg C_2$, this requires $S_1 \ll S_2$

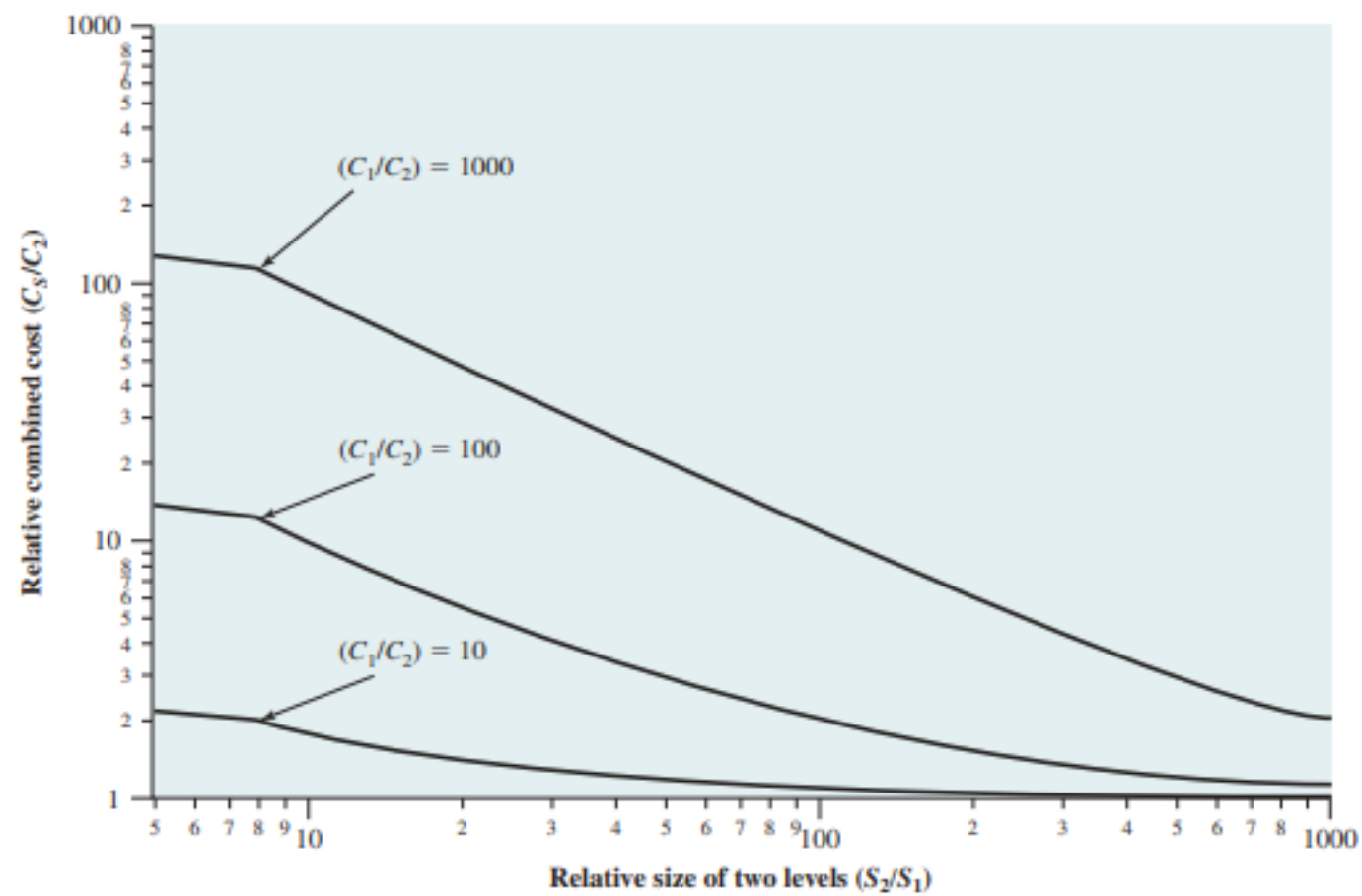


Figure 1.22 Relationship of Average Memory Cost to Relative Memory Size for a Two-Level Memory

Access time (T_s)

- For a two-level memory to provide a significant performance improvement, we need T_s approximately equal to T_1 .
- $\frac{T_1}{T_s} = \frac{1}{1 + (1-H) \cdot \frac{T_2}{T_1}}$
- $\frac{T_1}{T_s}$ is called **Access efficiency**
- **Let $r = \frac{T_2}{T_1}$ (ratio of access time of memory level 2 to level 1)**

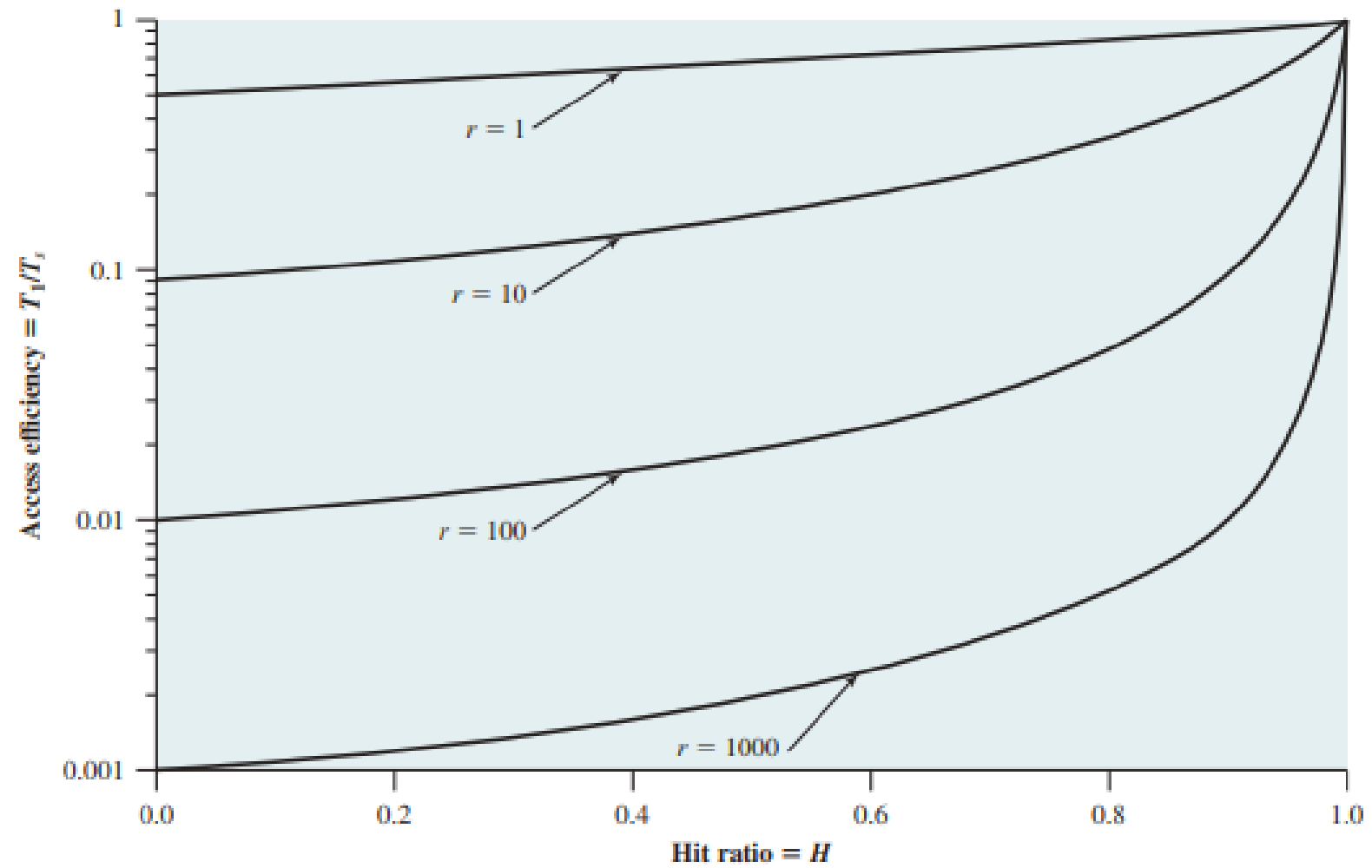


Figure 1.23 Access Efficiency as a Function of Hit Ratio ($r = T_2/T_1$)